

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Зачётная работа №3
по дисциплине «Алгоритмы и структуры данных»
на тему «КОМБИНИРОВАННЫЕ СТРУКТУРЫ ДАННЫХ И
СТАНДАРТНАЯ БИБЛИОТЕКА ШАБЛОНОВ»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинко П.Г.

Санкт-Петербург
2025

Содержание

Цель работы	3
Задание	3
Описание контейнера и используемых функций STL.....	3
Результаты работы программы.....	5
Оценка временной сложности	10
Текст программы.....	12
Выводы	18
Список используемых источников.....	19

Цель работы

Реализовать пользовательскую структуру данных для хранения множеств с поддержкой операций над ними.

Задание

Реализовать структуру данных на базе дерева двоичного поиска с хранимой высотой (ДДПв) для хранения множеств и поддержки операций над последовательностями.

Необходимо выполнить цепочку операций над множествами по выражению: $A \cap B \cup C \cup (D \oplus E)$

А также реализовать следующие дополнительные операции над последовательностями:

MERGE — слияние с сохранением дубликатов,

EXCL — исключение подпоследовательности,

CHANGE — замена части последовательности с указанной позиции.

Результаты всех операций должны быть отображены на экране, а реализованные алгоритмы — устойчивы к исключениям.

Описание контейнера и используемых функций STL

Контейнер реализован в виде пользовательского класса *DDP*, основанного на сбалансированном дереве двоичного поиска. В каждом узле дерева (*Node*) хранится:

- целочисленный ключ (*key*);
- указатели на левого и правого потомков (*l*, *r*);
- значение высоты поддерева (*h*), используемое для поддержания баланса.

Для обеспечения эффективной работы с множествами реализованы следующие базовые операции:

- *add(int k)* — вставка ключа в дерево и в последовательность;
- *removeOne(int k)* — удаление одного экземпляра ключа;

- *contains(int k)* — проверка наличия ключа в дереве;
- *cardinality()* — вычисление мощности множества;
- *unionWith, intersectWith, symDiffWith* — операции над множествами: объединение, пересечение и исключающее «или»;
- *MERGE, EXCL, CHANGE* — операции над последовательностями, включающие слияние, исключение подпоследовательности и замену части последовательности.

Для хранения порядка элементов используется дополнительная структура — вектор *std::vector<Node*> seq*, содержащий указатели на узлы дерева в порядке их добавления. Это позволяет реализовать работу с последовательностями на основе множества.

Также используются следующие стандартные компоненты библиотеки *STL*:

- *std::vector* — основной контейнер для хранения порядка узлов;
- *std::shuffle, std::random_device, std::mt19937* — генерация случайных перестановок и значений;
- *std::find_if, std::none_of* — алгоритмы поиска в векторе;
- *std::iota, std::sort* — генерация последовательностей и их упорядочивание;
- *std::unordered_set* — используется в операции симметрической разности для быстрой проверки наличия элемента;
- *std::ostream* — вывод последовательностей и визуализация дерева.

Программа поддерживает как операции над множествами, так и операции над последовательностями. Это достигается благодаря комбинированию дерева поиска с поддержкой баланса и дополнительного контейнера, обеспечивающего произвольный порядок хранения элементов.

Использование функций из *STL* обеспечивает лаконичность и эффективность реализации: переборы и модификации контейнеров выполняются стандартными алгоритмами, а операции над

последовательностями — средствами *std::vector* с прямым доступом к элементам.

Результаты работы программы

```

===== SET OPERATIONS =====
A:
4 9 10 1 13 8 0 5 7 14
BSTh(H=4 n=10) ---->
.....9.....
.....
.....4.....13.....
.....
....1.....7.....10.....14..
.....
..0.....5...8.....
.....

B:
14 4 12 8 3 1 9 2 7 13
BSTh(H=4 n=10) ---->
.....4.....
.....
.....2.....12.....
.....
....1.....3.....8.....14..
.....
.....7...9...13....
.....

C:
10 2 7 4 9 0 8 11 13 3
BSTh(H=4 n=10) ---->
.....7.....
.....
.....2.....9.....
.....
....0.....4.....8.....11..
.....
.....3.....10...13
.....

```

Рисунок 1 – Исходные множества *A, B, C*

```

D:
7 3 4 5 8 0 1 11 10 6
BSTh(H=4 n=10) ---->
.....4.....
.....
.....1.....7.....
.....
....0.....3.....5.....10..
.....
.....6...8...11
.....

E:
3 4 0 9 8 11 5 12 2 7
BSTh(H=4 n=10) ---->
.....8.....
.....
.....3.....11.....
.....
....0.....5.....9.....12..
.....
.....2...4...7.....
.....

```

Рисунок 2 – Исходные множества D и E

```

===== SET OPERATIONS =====
A:
Error: Sequence is empty

```

Рисунок 3 – Обработка вывода пустого множества

```

A ∩ B:
14 13 8 9 1 4 7
BSTh(H=4 n=7) ---->
.....9.....
.....
.....4.....13.....
.....
....1.....7.....14..
.....
.....8.....
.....

(A ∩ B) ∪ C:
3 14 10 7 8 9 1 2 13 0 4 11
BSTh(H=5 n=12) ---->
.....4.....
.....
.....1.....9.....
.....
....0.....2.....7.....13.....
.....
.....3.....8.....10.....14..
.....
.....11.....
.....

```

Рисунок 4 – Последовательное выполнение операций пересечения и объединения

```

D ⊕ E:
7 11 5 8 3 2 1 0 12 4 6 9 10
BSTh(H=4 n=13) ---->
.....6.....
.....
.....2.....10.....
.....
....1.....4.....8.....11..
.....
..0.....3...5...7...9.....12
.....

((A ∩ B) ∪ C) ∪ (D ⊕ E):
5 9 12 10 3 0 1 8 6 2 11 14 4 7 13
BSTh(H=5 n=15) ---->
.....4.....
.....
.....1.....9.....
.....
.....0.....2.....7.....13.....
.....
.....3.....5.....8.....11.....14..
.....
.....6.....10..12.....
.....

```

Рисунок 5 – Выполнение симметрической разности, объединения и получение итогового результата

```

===== SEQUENCE OPERATIONS =====
M1:
5 5 0 3 9 14 0 8 3 9
BSTh(H=3 n=6) ---->
.....5.....
.....
....3.....9...
.....
..0.....8...14
.....

M2:
0 5 4 12 8 2 14 12 1 11
BSTh(H=4 n=9) ---->
.....4.....
.....
.....1.....8.....
.....
....0.....2.....5.....12..
.....
.....11..14
.....

MERGE(M1,M2):
0 0 0 1 2 3 3 4 5 5 5 8 8 9 9 11 12 12 14 14
BSTh(H=4 n=11) ---->
.....3.....
.....
.....1.....9.....
.....
....0.....2.....5.....12..
.....
.....4...8...11..14
.....

```

Рисунок 6 – Выполнение операции MERGE

```

E1:
0 1 2 3 4 5 6 7 8
BSTh(H=4 n=9) ----->
.....3.....
.....
.....1.....5.....
.....
....0.....2.....4.....7...
.....
.....6...8.
.....

```

```

E2:
3 4 5
BSTh(H=2 n=3) ----->
....4...
.....
..3...5.
.....

```

```

E1 EXCL E2:
0 1 2 6 7 8
BSTh(H=3 n=6) ----->
.....6.....
.....
....1.....7...
.....
..0...2.....8.
.....

```

Рисунок 7 – Корректное выполнение EXCL без ошибок

```

E1:
0 1 2 3 4 5 6 7 8
BSTh(H=4 n=9) ----->
.....3.....
.....
.....1.....5.....
.....
....0.....2.....4.....7...
.....
.....6...8.
.....

```

```

E2:
3 6 5
BSTh(H=2 n=3) ----->
....5...
.....
..3...6.
.....

```

Error: Subsequence not found

Рисунок 8 – Обработка ошибки при выполнении EXCL

```

CH1:
10 4 6 12 12 9 14 6 2 6
BSTh(H=4 n=7) ----->
.....10.....
.....
.....6.....12.....
.....
....4.....9.....14..
.....
..2.....
.....

CH1 CHANGE from index 2 by 83 56 38
10 4 83 56 38 9 14 6 2 6
BSTh(H=4 n=9) ----->
.....10.....
.....
.....6.....56.....
.....
....4.....9.....38.....83..
.....
..2.....14.....
.....

```

Рисунок 9 – Корректное выполнение CHANGE без ошибок

```

CH1:
6 4 3 9 6 14 8 2 0 6
BSTh(H=4 n=8) ----->
.....6.....
.....
.....3.....9.....
.....
....2.....4.....8.....14..
.....
..0.....
.....

CH1 CHANGE from index 11 by 90 62 60
Error: Position is outside the sequence

```

Рисунок 10 – Обработка ошибки при выполнении CHANGE

Оценка временной сложности

Реализованная структура данных *DDP* основана на сбалансированном дереве двоичного поиска с хранением высоты узлов. Благодаря использованию автоматической балансировки, основные операции над множеством выполняются за логарифмическое время в среднем случае.

Вставка (*add*) и удаление (*removeOne*) элемента:

- Сложность: $O(\log n)$ — благодаря поддержанию баланса высот поддеревьев.
- В худшем случае (если дерево становится несбалансированным, например, при ручной модификации последовательности) — $O(n)$.

Поиск элемента (*contains*):

- Сложность: $O(\log n)$ — за счёт бинарного поиска по дереву.

Объединение, пересечение и исключаящее «или» (*unionWith*, *intersectWith*, *symDiffWith*):

- Реализованы на основе внутреннего обхода дерева (*inorder*), с последующей вставкой элементов.
- Средняя сложность: $O(n \cdot \log n)$ — для вставки n элементов в сбалансированное дерево.
- Для операций исключаящего «или» используется *std::unordered_set*, что позволяет добиться $O(1)$ времени проверки принадлежности элемента, тем самым улучшая общую производительность.

MERGE (слияние двух последовательностей):

- Включает сортировку и вставку всех элементов.
- Сложность: $O((n + m) \cdot \log(n + m))$, где n , m — размеры двух последовательностей.

EXCL (исключение подпоследовательности):

- Поиск подпоследовательности осуществляется через линейный просмотр (*std::find_if*), а затем выполняется перестроение дерева.
- В худшем случае (подпоследовательность не найдена): $O(n^2)$.
- В среднем случае: $O(n \cdot \log n)$.

CHANGE (замена с позиции):

- Удаление и вставка вектора: $O(k + m)$, где k — число удаляемых элементов, m — вставляемых.
- Перестроение дерева заново: $O(n \cdot \log n)$.

rebuildTree() и *rebuildSeqRandom()* — перестраивают дерево и вектор *seq*. В обоих случаях сложность: $O(n \cdot \log n)$ — так как осуществляется вставка всех n ключей заново.

Благодаря использованию сбалансированного дерева и стандартных алгоритмов *STL*, реализация обеспечивает приемлемую временную сложность

для всех операций, типичных при работе с множествами и последовательностями. Большинство операций имеют логарифмическую или квазилинейную ($O(n \cdot \log n)$) сложность, а при осторожном использовании *EXCL* и *CHANGE* удаётся избежать худших случаев.

Текст программы

```
#include <bits/stdc++.h>
#include <stdexcept>

class DDP {
    struct Node {
        int key;
        Node *l, *r;
        int h;

        explicit Node(const int k): key(k), l(nullptr), r(nullptr), h(1) {
        }
    };

    Node *root = nullptr;
    std::vector<Node *> seq;

    static int height(const Node *n) { return n ? n->h : 0; }

    static void upd(Node *n) {
        if (!n) return;
        n->h = std::max(height(n->l), height(n->r)) + 1;
    }

    static Node *rotR(Node *y) {
        Node *x = y->l;
        Node *t = x->r;
        x->r = y;
        y->l = t;
        upd(y);
        upd(x);
        return x;
    }

    static Node *rotL(Node *x) {
        Node *y = x->r;
        Node *t = y->l;
        y->l = x;
        x->r = t;
        upd(x);
        upd(y);
        return y;
    }

    static int bal(const Node *n) { return n ? height(n->l) - height(n->r) : 0; }

    static Node *insert(Node *n, int k, Node **out) {
        if (!n) {
            *out = new Node(k);
            return *out;
        }
    }
}
```

```

    if (k < n->key) n->l = insert(n->l, k, out);
    else if (k > n->key) n->r = insert(n->r, k, out);
    else {
        *out = n;
        return n;
    }
    upd(n);
    const int bf = bal(n);
    if (bf > 1 && k < n->l->key) return rotR(n);
    if (bf < -1 && k > n->r->key) return rotL(n);
    if (bf > 1 && k > n->l->key) {
        n->l = rotL(n->l);
        return rotR(n);
    }
    if (bf < -1 && k < n->r->key) {
        n->r = rotR(n->r);
        return rotL(n);
    }
    return n;
}

static Node *minVal(Node *n) {
    while (n->l) n = n->l;
    return n;
}

static Node *erase(Node *n, const int k) {
    if (!n) return nullptr;
    if (k < n->key) n->l = erase(n->l, k);
    else if (k > n->key) n->r = erase(n->r, k);
    else {
        if (!n->l || !n->r) {
            Node *t = n->l ? n->l : n->r;
            delete n;
            return t;
        }
        const Node *s = minVal(n->r);
        n->key = s->key;
        n->r = erase(n->r, s->key);
    }
    upd(n);
    const int bf = bal(n);
    if (bf > 1 && bal(n->l) >= 0) return rotR(n);
    if (bf > 1 && bal(n->l) < 0) {
        n->l = rotL(n->l);
        return rotR(n);
    }
    if (bf < -1 && bal(n->r) <= 0) return rotL(n);
    if (bf < -1 && bal(n->r) > 0) {
        n->r = rotR(n->r);
        return rotL(n);
    }
    return n;
}

static void inorder(const Node *n, std::vector<int> &v) {
    if (!n) return;
    inorder(n->l, v);
    v.push_back(n->key);
    inorder(n->r, v);
}

```

```

void rebuildTree() {
    root = nullptr;
    for (Node *p: seq) {
        Node *ref = nullptr;
        root = insert(root, p->key, &ref);
        p = ref;
    }
}

void rebuildSeqRandom() {
    std::vector<int> keys;
    inorder(root, keys);
    std::shuffle(keys.begin(), keys.end(), std::mt19937{std::random_device{}}());
    seq.clear();
    for (const int k: keys) {
        Node *ref = nullptr;
        insert(root, k, &ref);
        seq.push_back(ref);
    }
}

public:
    DDP() = default;

    explicit DDP(const std::vector<int> &v) { for (const int k: v) add(k); }

    void add(const int k) {
        Node *ref = nullptr;
        root = insert(root, k, &ref);
        seq.push_back(ref);
    }

    void removeOne(const int k) {
        const auto it = std::find_if(seq.begin(), seq.end(), [&](const Node *n) {
return n->key == k; });
        if (it == seq.end()) return;
        seq.erase(it);
        if (std::none_of(seq.begin(), seq.end(), [&](const Node *n) { return n->key
== k; }))) root = erase(root, k);
    }

    size_t cardinality() const {
        std::vector<int> v;
        inorder(root, v);
        return v.size();
    }

    bool contains(const int k) const {
        const Node *c = root;
        while (c) {
            if (k == c->key) return true;
            c = k < c->key ? c->l : c->r;
        }
        return false;
    }

    void unionWith(const DDP &o) {
        std::vector<int> ks;
        inorder(o.root, ks);
        for (const int k: ks) {

```

```

        Node *r = nullptr;
        root = insert(root, k, &r);
    }
    rebuildSeqRandom();
}

void intersectWith(const DDP &o) {
    std::vector<int> ks;
    inorder(root, ks);
    for (const int k: ks) if (!o.contains(k)) root = erase(root, k);
    rebuildSeqRandom();
}

void symDiffWith(const DDP &o) {
    std::vector<int> a;
    inorder(root, a);
    std::vector<int> b;
    inorder(o.root, b);
    const std::unordered_set<int> sb(b.begin(), b.end());
    for (int k: a) if (sb.count(k)) root = erase(root, k);
    for (const int k: b)
        if (!contains(k)) {
            Node *r = nullptr;
            root = insert(root, k, &r);
        }
    rebuildSeqRandom();
}

static DDP MERGE(const DDP &a, const DDP &b) {
    std::vector<int> keys;
    keys.reserve(a.seq.size() + b.seq.size());
    for (const auto *p: a.seq) keys.push_back(p->key);
    for (const auto *p: b.seq) keys.push_back(p->key);
    std::sort(keys.begin(), keys.end());
    DDP res;
    for (const int k: keys) res.add(k);
    return res;
}

void EXCL(const DDP &sub) {
    const size_t n = sub.seq.size();
    if (n == 0 || n > seq.size())
        throw std::runtime_error("Subsequence not found");

    for (size_t i = 0; i + n <= seq.size(); ++i) {
        bool ok = true;
        for (size_t j = 0; j < n; ++j)
            if (seq[i + j]->key != sub.seq[j]->key) {
                ok = false;
                break;
            }
        if (ok) {
            seq.erase(seq.begin() + i, seq.begin() + i + n);
            rebuildTree();
            return;
        }
    }
    throw std::runtime_error("Subsequence not found");
}

void CHANGE(const size_t pos, const DDP &repl) {

```

```

        if (pos >= seq.size())
            throw std::out_of_range("Position is outside the sequence");

        std::vector<int> keys;
        keys.reserve(seq.size());
        for (const auto *p: seq) keys.push_back(p->key);

        const size_t eraseLen = std::min(repl.seq.size(), keys.size() - pos);
        if (eraseLen == 0 && repl.seq.empty())
            throw std::runtime_error("Nothing to change: both erase length and
replacement are empty");
        keys.erase(keys.begin() + pos, keys.begin() + pos + eraseLen);
        std::vector<int> replKeys;
        replKeys.reserve(repl.seq.size());
        for (const auto *p: repl.seq) replKeys.push_back(p->key);
        keys.insert(keys.begin() + pos, replKeys.begin(), replKeys.end());
        clear();
        for (const int k: keys) add(k);
    }

    void printSeq(std::ostream &os = std::cout) const {
        if (seq.empty()) throw std::runtime_error("Sequence is empty");
        for (const auto *p: seq) os << p->key << ' ';
        os << '\n';
    }

    void printTree(std::ostream &os = std::cout) const {
        const int h = height(root);
        if (!h) throw std::runtime_error("Tree is empty");
        const int w = (1 << h) * 2, rows = h * 2;
        std::vector<std::string> mat(rows, std::string(w, '.'));
        fillMatrix(root, w / 2, 0, w / 2, mat);
        os << "BSTh(H=" << h << " n=" << cardinality() << ") ----->\n";
        for (auto &ln: mat) os << ln << '\n';
    }

private:
    static void fillMatrix(const Node *n, const int col, const int row, const int
off, std::vector<std::string> &m) {
        if (!n)return;
        const std::string s = std::to_string(n->key);
        for (size_t i = 0; i < s.size(); ++i)m[row][col + i] = s[i];
        const int gap = off / 2;
        if (n->l)fillMatrix(n->l, col - gap, row + 2, gap, m);
        if (n->r)fillMatrix(n->r, col + gap, row + 2, gap, m);
    }

    void clear() {
        std::function<void(Node *)> del = [&](const Node *n) {
            if (!n)return;
            del(n->l);
            del(n->r);
            delete n;
        };
        del(root);
        root = nullptr;
        seq.clear();
    }

public:
    ~DDP() { clear(); }

```

```

private:
    static std::vector<int> sampleUnique(const int cnt, const int univ) {
        if (univ < cnt) throw std::invalid_argument("universe < power");
        std::vector<int> v(univ);
        std::iota(v.begin(), v.end(), 0);
        std::shuffle(v.begin(), v.end(), std::mt19937{std::random_device{}}());
        v.resize(cnt);
        return v;
    }

public:
    static DDP genUniqueSet(const int power, const int universe) {
        DDP o;
        for (const int k: sampleUnique(power, universe)) o.add(k);
        return o;
    }

    static DDP genSequence(const int len, const int universe) {
        DDP o;
        std::mt19937 rng{std::random_device{}}();
        std::uniform_int_distribution<int> dist(0, universe - 1);
        for (int i = 0; i < len; ++i) o.add(dist(rng));
        return o;
    }
};

int main() {
    using std::cout;

    auto safePrint = [&](const std::string &label, const DDP &ds) {
        cout << label;
        try {
            ds.printSeq();
            ds.printTree();
        } catch (const std::exception &ex) {
            cout << "Error: " << ex.what() << '\n';
        }
    };

    cout << "===== SET OPERATIONS =====\n";
    DDP A = DDP::genUniqueSet(10, 15);
    DDP B = DDP::genUniqueSet(10, 15);
    DDP C = DDP::genUniqueSet(10, 15);
    DDP D = DDP::genUniqueSet(10, 15);
    DDP E = DDP::genUniqueSet(10, 15);

    safePrint("A:\n", A);
    safePrint("\nB:\n", B);
    safePrint("\nC:\n", C);
    safePrint("\nD:\n", D);
    safePrint("\nE:\n", E);

    DDP T = A;
    T.intersectWith(B);
    safePrint("\nA n B:\n", T);

    T.unionWith(C);
    safePrint("\n(A n B) U C:\n", T);
}

```

```

DDP tmp = D;
tmp.symDiffWith(E);
safePrint("\nD  $\oplus$  E:\n", tmp);

T.unionWith(tmp);
safePrint("\n((A  $\cap$  B)  $\cup$  C)  $\cup$  (D  $\oplus$  E):\n", T);

cout << "\n===== SEQUENCE OPERATIONS =====";
DDP M1 = DDP::genSequence(10, 15);
DDP M2 = DDP::genSequence(10, 15);
safePrint("\nM1:\n", M1);
safePrint("\nM2:\n", M2);
DDP M = DDP::MERGE(M1, M2);
safePrint("\nMERGE(M1,M2):\n", M);

DDP E1({0, 1, 2, 3, 4, 5, 6, 7, 8});
DDP E2({3, 4, 5});
safePrint("\nE1:\n", E1);
safePrint("\nE2:\n", E2);
try {
    E1.EXCL(E2);
    safePrint("\nE1 EXCL E2:\n", E1);
} catch (const std::exception &ex) {
    cout << "\nError: " << ex.what() << '\n';
}

DDP CH1 = DDP::genSequence(10, 15);
DDP CH2 = DDP::genSequence(3, 100);
constexpr int ind = 2;
safePrint("\nCH1:\n", CH1);
cout << "\nCH1 CHANGE from index " << ind << " by ";
CH2.printSeq();
try {
    CH1.CHANGE(ind, CH2);
    safePrint("", CH1);
} catch (const std::exception &ex) {
    cout << "Error: " << ex.what() << '\n';
}

return 0;
}

```

Выводы

В рамках лабораторной работы была разработана и реализована структура данных *DDP* — пользовательский контейнер на основе сбалансированного дерева двоичного поиска с хранимой высотой. Контейнер предназначен для хранения множеств и поддержки операций как над множествами, так и над последовательностями элементов.

Были реализованы и протестированы основные операции над множествами:

- добавление, удаление и поиск элементов;

- вычисление мощности множества;
- объединение, пересечение и исключающее «или».

Кроме того, реализована поддержка работы с последовательностями, что позволило выполнить такие операции, как:

- слияние двух последовательностей с сохранением дубликатов (*MERGE*);
- исключение подпоследовательности (*EXCL*);
- замена части последовательности на другую с указанной позиции (*CHANGE*).

Для хранения порядка элементов в контейнере используется дополнительный вектор указателей на узлы дерева, что обеспечивает возможность управления последовательностью, независимо от структуры самого дерева.

Реализация показала высокую эффективность: основные операции над множествами выполняются в среднем за логарифмическое время, а операции с последовательностями — за линейное или квазилинейное время. Использование стандартных алгоритмов *STL* и внутренних итераторов делает код компактным и устойчивым к исключениям.

Визуализация состояния дерева и последовательности на каждом этапе операций позволяет убедиться в корректности работы алгоритмов и наглядно демонстрирует изменения, происходящие в структуре данных.

Список используемых источников

1. Колинко П. Г. Пользовательские контейнеры: учебно-метод. пособие. СПб.: СПбГЭТУ «ЛЭТИ», 2025. 64 с. (вып.2502)